

Mapeamento visual de arquiteturas de software C++ orientado a objetos

Hugo Rodrigues Torquato^{1*}; Heinrich da Solidade Santos²

¹ Engenheiro de Software II. Mestre. Rua lavras 480 – São Pedro; 3033010 Belo Horizonte, Minas Gerais, Brasil.

². USP/ESALQ. Doutor. Av. Pádua Dias, 11 - Agronomia; 13418-900 Piracicaba, São Paulo, Brasil

*autor correspondente: hugortorquato@gmail.com

Mapeamento visual de arquiteturas de software C++ orientado a objetos

Resumo

O conhecimento da própria base de código foi considerado um fator fundamental tanto para a tomada de decisão quanto para a integração de novos engenheiros às equipes. Em diversas ocasiões, a falta de domínio da própria implementação é uma barreira encontrada pelas empresas. Diante desse cenário, o presente projeto propôs a criação de uma API capaz de avaliar estaticamente uma base de código e apresentar informações de uma maneira simplificada e intuitiva. O escopo foi definido para identificação da relação de classes em projetos orientados a objetos na linguagem C++. O desenvolvimento contemplou etapas iniciais de compiladores, como a construção de um "scanner" e um "parser". Mas abordou a definição e implementação da infraestrutura para acesso ao código fonte por meio das APIs do GitHub. Também foram abordadas decisões sobre a arquitetura do software com a escolha de diversos padrões de projeto, como o "Singleton", "Strategy" e "Visitor", de modo a garantir escalabilidade e flexibilidade da solução. A confiabilidade foi assegurada por meio de testes unitários e funcionais, baseados em Test-Driven Development. O resultado obtido foi uma API que retorna, em formato JSON, a relação entre as classes do projeto avaliado. O funcionamento foi validado com exemplos de arquiteturas de herança simples e complexas, incluindo herança múltipla, em cascata e em diamante, bem como com o próprio código desenvolvido para esse estudo. Constatou-se que a ferramenta fornece resultados eficazes e abriu perspectivas para trabalhos futuros, como a análise de novos elementos do código e suporte a outras linguagens de programação.

Palavras-chave: Compiladores; Análise estática; C++; API; Test-Driven Development

Introdução

No contexto de programação orientada a objetos, as abstrações são fundamentais para a organização e atribuição de responsabilidades. A compreensão da lógica é simples quando os assuntos são comumente conhecidos e/ou com poucos níveis de abstração. Entretanto, o código se torna mais complexo e, por consequência, demanda mais esforço e tempo para compreendê-lo quando se trata de uma base de código extensa, desenvolvida durante vários anos, por vários profissionais e com foco em um conhecimento específico. Em uma ótica empresarial, este cenário implica maiores investimentos para que novos engenheiros sejam incorporados à equipe por causa da complexidade, e adiciona um nível de abstração extra nas tomadas de decisão relacionadas ao desenvolvimento do software por não ter uma visão geral do todo. Sob essa perspectiva, visualizar a organização de uma base de código clara e intuitiva agrega valor e praticidade, principalmente na rotina das empresas desenvolvedoras de software.

A visualização e organização de informações de um software auxiliam no seu gerenciamento, o que, de forma secundária, reflete na maior manutenibilidade do produto. Martin (2009) ilustra essa dinâmica ao comparar a produtividade pelo tempo de existência do software: ao passo que a extensão e complexidade do código aumentam, especialmente sem planejamento, as chances de se transformar em um emaranhado de soluções

improvisadas também aumentam. Nesse cenário, o autor argumenta que a produtividade para desenvolver novas funcionalidades e a manutenção do código declinam drasticamente a médio e longo prazo, quando não gerenciados corretamente, tornando-se quase impraticáveis e economicamente inviáveis.

O ponto de partida para projetos dessa natureza contempla a análise de código fonte, tema de investigação consolidado para diversas áreas na computação: a otimização de compilador, abordado por Dean (1995), que expõe os benefícios ao realizar análises de hierarquia de classes como método para resolver ambiguidades em invocações de funções virtuais; análises de qualidade de software, abordado por Ceconello (2016), apresenta alguns dos problemas recorrentes na programação orientada a objetos; e estudos até mesmo no ramo da avaliação do quão eficiente estão os modelos de inteligência artificial baseados em linguagens (LLMs) quando comparado à programação funcional com orientada a objetos, abordado por Wang (2024), que evidencia a necessidade de melhorias.

Inúmeras aplicabilidades, soluções e produtos foram propostos com a finalidade de aprimorar a experiência durante e após o desenvolvimento de software. Silva (2023) detalha os benefícios gerais das ferramentas que podem ser integradas em diversos momentos no ciclo de vida do software, desde a etapa de codificação até a validação do produto final. Como exemplos temos: o SonarQube, uma plataforma de análise de código com foco em qualidade e cobertura de testes; o Clang Static Analyzer, uma ferramenta de análise nativa do LLVM aplicada ao C e C++; e AddressSanitizer, como ferramenta para identificar erros de memória em tempo de execução.

Frente ao exposto, ferramentas de visualização e organização de uma base de código são bastante requisitadas em projetos de software. Por consequência, é tema de pesquisas tanto no âmbito acadêmico quanto no desenvolvimento de novos produtos e soluções no setor privado. A literatura relacionada a este projeto de pesquisa está associada à investigação feita em nível de compiladores e nos ambientes de desenvolvimento integrado (IDEs), com o intuito de auxiliar o programador na escrita e na otimização da compilação de código.

O objetivo deste trabalho foi propor o mapeamento estruturado para facilitar discussões e a tomada de decisão não tem sido amplamente abordado e/ou mencionado em relevantes estudos e produtos. Sendo assim, faz-se relevante arquitetar um software capaz de realizar uma análise estática em códigos baseados em programação orientada a objetos, extrair informação sobre a organização das classes e expor, de maneira simples e direta, informações estruturadas acerca do código em questão.

Material e Métodos

O presente trabalho fundamenta-se na análise de código que, conforme apresentado pela literatura, pode ser conduzida por uma abordagem tanto estática quanto dinâmica. Um comparativo é apresentado por Silva (2023), que ressalta tanto os aspectos positivos quanto os reveses de cada estratégia. O autor enfatiza os benefícios alcançados quando utilizados em conjunto. No contexto deste projeto de pesquisa, a análise estática revelou-se uma opção promissora por depender unicamente do código, sem a necessidade de comprá-lo. Essa característica torna o projeto proposto agnóstico em relação à linguagem de programação e permite avaliar a intenção original do desenvolvedor ao examinar o código antes de eventuais otimizações dos compiladores.

As etapas iniciais do projeto concentraram-se na implementação de elementos típicos de um compilador, com início no mapeamento do código fonte e evoluíram para a organização das informações no formato de "tokens". Essa categoria de dados constitui a entrada para o processo de "parsing", responsável por agrupar os "tokens" em expressões sintáticas mais abrangentes e viabilizar a construção das chamadas árvores de sintaxe. Essas, por sua vez, possibilitam a análise do código de acordo com as especificidades da linguagem de programação utilizada. As informações necessárias para o projeto de pesquisa proposto foram coletadas nesta etapa, com ênfase na estrutura da árvore de sintaxe.

Embora existam outras etapas em um projeto de compilador, os resultados por elas gerados não agregam informações relevantes a este projeto. Ademais, Nystrom (2020) apresenta uma visão detalhada de todo o processo de desenvolvimento de um compilador, explorando as etapas: otimização do código fonte; tradução para linguagem de baixo nível mais próxima de máquina e compilação conforme a escolha do ambiente de execução do software. A Figura 1 apresenta, de forma simplificada, o fluxo completo das etapas de desenvolvimento de um compilador, sendo cada uma detalhada por Nystrom (2020). As etapas que interagem com o escopo deste trabalho são: "scanner", "tokens", "parsing" e "analysis".

Os autores Nystrom (2020), Aho (2007) e Cooper (2012) foram as referências principais para fundamentar o tema compiladores. Com foco nas etapas relevantes para este trabalho, temos o mapeamento dos caracteres de um código como a primeira das três etapas fundamentais. Nesta fase inicial, a performance é crucial, uma vez que requer uma análise sequencial de todos os caracteres dos arquivos fornecidos ao sistema. Todas as informações precisam ser categorizadas de forma eficiente para possibilitar o agrupamento dos caracteres em "tokens", que constituem uma abstração de informações úteis para o

interpretador. Em termos simplificados, o menor agrupamento de caracteres com significado, incluindo o tratamento de ambiguidades lexicais como o operador aritmético de multiplicação, por exemplo, que também significa uma operação de ponteiros em C++.

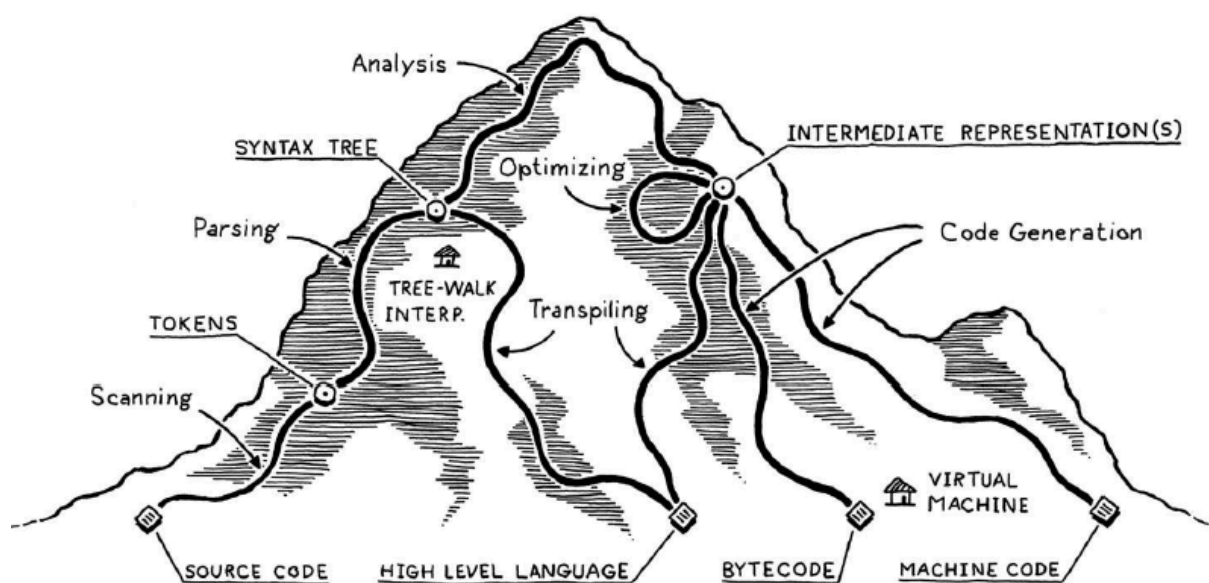


Figura 1. Ilustração dos possíveis caminhos de implementação de compiladores.

Fonte: Nystrom (2020)

Além de percorrer todos os caracteres com a intenção de identificar os "tokens", a etapa do "scanner" também é responsável por associar essas informações à linha correspondente no arquivo fonte. Em termos de compiladores, essa rastreabilidade vai enriquecer o apontamento e a disponibilidade de informações em caso de erros. Já no escopo deste projeto, a rastreabilidade de "tokens" foi fundamental na investigação dos comportamentos inesperados durante a implementação e também pode ser recuperada no resultado final caso essa informação tenha valor para o usuário.

O processo de identificação dos "tokens" segue então para a identificação dos lexemes, definidos como grupo de caracteres conhecidos, mas ainda sem significado léxico. O processo consiste em, segundo tradução livre de Nystrom (2020), "um grande switch case com delírios de grandeza". A implementação contempla um processo detalhado de mapeamento de cada caractere com significado no C++ e a criação dos "tokens" com base na igualdade deste caractere, já mapeado, com o que está sendo avaliado pelo algoritmo. Alguns grandes grupos foram identificados como: os lexemas unários, que necessitam apenas de um caractere para possuir um significado, como o operador de atribuição de valor para uma variável "="; os lexemes binários, que já contemplam dois caracteres para obter um significado, como o operador de comparação de igualdade "=="; as palavras reservadas

da linguagem, como "if" e "else"; e as demais representações numerais e de identificadores de variáveis.

O grande ponto desse switch case é identificar cada um dos possíveis lexemes resolvendo as ambiguidades lexicais, como a diferença entre o operador de atribuição "=" e o de comparação de igualdade "==" apresentados como exemplo. Para esses casos, é feita uma avaliação considerando o próximo caractere da pilha a ser analisada; se o caractere atual for "=" e o próximo também for "=", é criado o token de comparação de igualdade, caso contrário, o de atribuição. A técnica implementada é descrita como olhar para frente (look ahead) e a utilizada considerou apenas um caractere à frente, o que torna a "scanner" com uma complexidade $O(n)$ que expande de acordo com o tamanho do arquivo a ser analisado.

Um exemplo, mas que ilustra algumas das regras utilizadas para a criação dos "tokens", é os comentários que identificam um determinado conjunto de caracteres, "//" no C++, e consomem todos os caracteres até chegar ao final da linha. De forma análoga, as strings também identificam o carácter inicial " " e consomem todos os caracteres subsequentes até encontrar outro carácter semelhante, criando assim o token associado à string, sendo o lexeme todo o conteúdo de caracteres contido entre os delimitadores.

Uma vez catalogado, um objeto token é criado para encapsular o conteúdo e demais informações relacionadas a esse trecho de código, metadados como a definição de linha, coluna e o próprio arquivo em que está definido. Cada objeto token, após ser criado, é adicionado a um container que agrupa todos os "tokens" já identificados e sua utilização é na próxima etapa do processamento do código, o "parser". O processo é ilustrado pela Figura 2 que começa com uma simples implementação que é convertida para um container de caracteres em sequência que contém todos os lexemes identificados. O "scanner" então começa no primeiro elemento e faz a tokenização elemento por elemento com o intuito de agrupar os caracteres na menor representação léxica possível, com significado e tratando as ambiguidades. Uma vez agrupados, os "tokens" são criados e separados para uso posterior. Ponto interessante é que esta é a última etapa do projeto que vai interagir, diretamente, com o carácter extraído do código fonte. As etapas subsequentes são processamentos e manipulações com o objeto token associado ao grupo de caracteres.

A segunda fase do processamento é realizada pelo "parser". Este componente é responsável por determinar se a sequência de "tokens", gerada pelo mapeamento do código, constitui uma sentença válida de acordo com a linguagem alvo. Ao final de todas as validações e processamentos, insere a informação do token em uma árvore de sintaxe que consiste em uma representação para a relação entre os símbolos. Dentre as possíveis implementações do "parser", algoritmos como top-down e bottom-up se destacam pela sua robustez e eficiência, mas diversas outras técnicas foram e são criadas com o intuito de

solucionar situações específicas. Então, a escolha do modelo a ser utilizado está relacionada com sua aplicação.

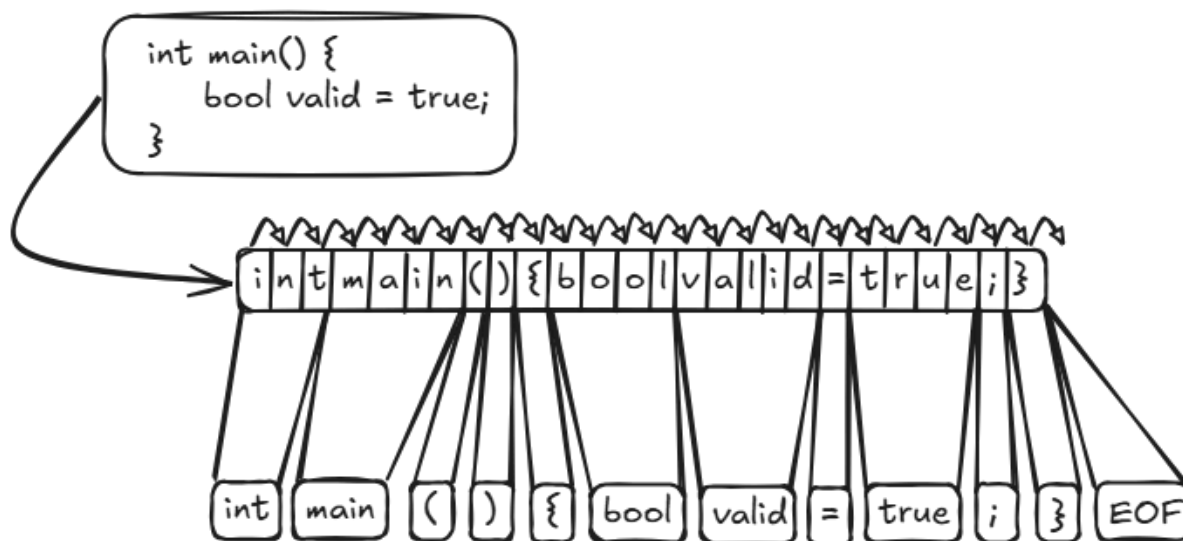


Figura 2. Ilustração do processo de tokenização de um código.

Fonte: Dados originais da pesquisa

No escopo deste projeto, foi adotado o modelo top-down, que analisa recursivamente do topo para o final com previsão. Esse fluxo parte do símbolo inicial da gramática e tenta expandi-la por meio das chamadas recursivas até chegar ao ponto em que não é mais possível, como uma regra sem recursão. Agora, a seleção da regra associada é feita com o padrão lookahead, olhar à frente, de 1 token e com isso os nós mais profundos da árvore são retomados para formar os nós das camadas intermediárias.

Um primeiro exemplo em linguagem natural é a frase “Eu vou comer um salgado”, em que pode ser construída duas regras para descrever o fato de que “estou interessado em comer uma comida”, qual comida? “salgado”, mas pode ser substituído por outra. Logo a construção da primeira regra, nomeada de regra fala, é “Eu vou comer uma <COMIDA>” e a segunda regra é que <COMIDA> pode assumir qualquer contexto de alimento. Com essas duas regras podemos, recursivamente, parsear a frase inicial para qualquer tipo de alimento. Para ler as regras definidas na eq.1 que apresenta as regras resultantes dessa estrutura, temos o nome da regra seguido dos valores que podem assumir, contendo as respectivas regras recursivas.

Regra fala → “Eu vou comer um <Regra comida>” (1)

Regra comida → salgado | pastel | pão de queijo

Em que Regra fala corresponde a uma sentença fixa que inclui uma parte variável, a comida; e Regra comida define as possíveis opções que podem preencher essa variação, sendo neste exemplo: salgado, pastel ou pão de queijo.

Agora, um exemplo prático para consolidar esse processo é a elaboração de expressões. A construção da árvore de sintaxe livre nesse caso consiste em categorizar os "tokens" em grupos de expressões, sejam elas expressões binárias que contemplam dois operandos e um operador, expressões unárias que consistem em um operador e um operando, expressões literais que consistem apenas em um operando como números, variáveis ou booleanos, e expressões de agrupamento que representam uma expression qualquer contida dentro de parêntesis. Sim, uma expression, que no caso é a classe base para todas as outras especializações. Sem adentrar em detalhes de implementação, a Figura 3 apresenta uma visão de alto nível da arquitetura de classes para a etapa de criação da árvore de sintaxe. Apesar do escopo reduzido do projeto, este é expansível caso necessário. O conceito inicial da arquitetura consiste em que todos os tipos de nós, possíveis de serem inseridos na árvore, tenham uma mesma classe base, no caso, o `CommonParserType`. Esta classe armazena informações gerais, mas permite uma conversão dinâmica de acordo com o token que está sendo processado no momento.

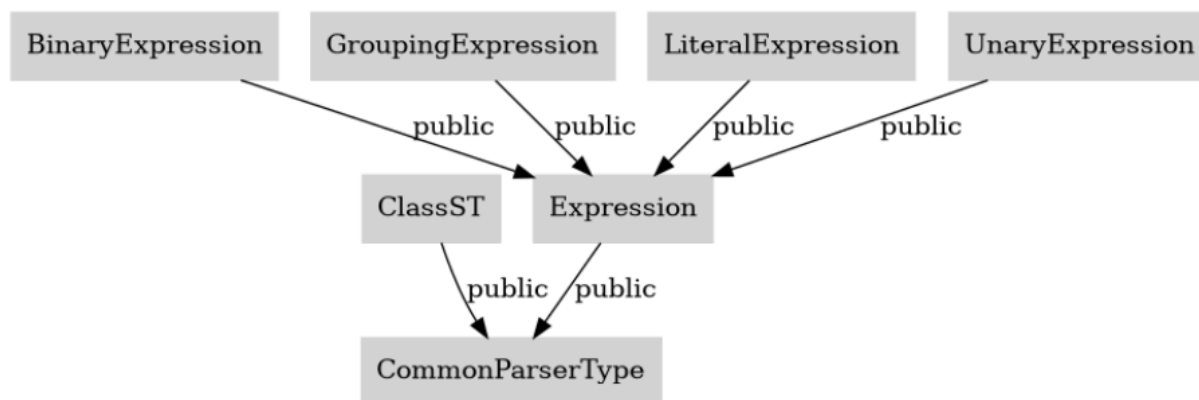


Figura 3. Hierarquia entre classes que representam objetos identificados pelo "parser"

Fonte: Dados originais da pesquisa

Ainda sobre a Figura 3, o grande foco deste projeto é a estrutura de classes representada pela classe `ClassST`, mas para compreender sua implementação precisamos consolidar o funcionamento do "parser", essa explicação será retomada posteriormente.

Retomando o exemplo de expressões, o foco está associado à subclasse `expressions` que, ainda, é uma camada de abstração, mas representa as especializações literais, unárias, binárias ou agrupamentos. Logo, é considerado como o ponto de entrada

para a análise recursiva do "parser" para qualquer conjunto de "tokens" que representam uma expressão. De forma mais teórica, essas definições contemplam a gramática livre de contexto que descreve como literais, operadores unários, binários e agrupamentos são organizados de acordo com as seguintes regras, presentes na eq.2:

$$\begin{aligned} \text{Expressions} &\rightarrow \text{literal} \mid \text{unary} \mid \text{binary} \mid \text{grouping} \\ \text{Literal} &\rightarrow \text{NUMBER} \mid \text{STRING} \mid \text{"true"} \mid \text{"false"} \mid \text{"nil"} \\ \text{Grouping} &\rightarrow \text{"(" expression ")"} \\ \text{Unary} &\rightarrow \text{" - " | "!" } \text{expression} \\ \text{Binary} &\rightarrow \text{expression operator expression} \\ \text{Operator} &\rightarrow \text{" == " | " != " | " < " | " <= " | " > " | " >= " | " + " | " - " | " * " | " / " } \end{aligned} \quad (2)$$

Em que Expressions corresponde ao ponto de entrada para a análise da expressão, podendo ser literal, unária, binária ou de agrupamento; Literal corresponde aos valores atômicos como números, strings e booleanos (true, false, nil); Grouping corresponde a expressão contida entre parênteses; Unary corresponde ao operador unário (- ou !) aplicado a uma expressão; Binary corresponde a uma expressão formada por dois operandos e um operador; e Operator que corresponde a um conjunto de operadores aritméticos e relacionais (==, !=, <, <=, >, >=, +, -, *, /).

Ao utilizar as regras em um exemplo concreto, temos a expressão $(1+2^{*}-(3-4))$ e a Figura 4 ilustrando o seu respectivo "parser". Primeiro é apresentada a lista de "tokens" que o conjunto de caracteres do código fonte forma, saída do componente "scanner" e entrada para o "parser". Uma vez iniciados, os elementos são processados de acordo com sua ordem de precedência, com o primeiro passo sendo adentrar na expressão da classe agrupamento, que por sua vez pode ser considerada como uma operação binária. O segundo passo é responsável por segregar dois operandos que estão separados por uma operação de adição, o primeiro operando é o literal numérico 1 e o segundo é outra expressão que será processada no passo 3. Esse também consiste em outra operação binária, mas com operador de multiplicação entre o literal numérico 2 e outra expressão ao lado direito. O passo 4 exemplifica a classificação de uma operação unária, em que o sinal negativo é introduzido antes da expressão de grupamento, alternando o sinal do resultado deste grupamento. O passo 5 faz outra operação de grupamento em que é analisada a expressão interna aos parênteses que, por sua vez, é uma operação binária. No passo 6, avaliam-se dois literais numéricos que são considerados as folhas finais dessa árvore de sintaxe.

Essa é uma maneira simples e robusta de criar um "parser" utilizando a técnica de análise de descendência recursiva. É considerado um top-down "parser" por começar pelo nó do topo da regra gramatical até atingir as folhas dessas regras, percorrendo a entrada de dados da esquerda para a direita. É estruturado com base em procedimentos mutuamente recursivos em que as regras definidas permitem combinações entre si, como o exemplo das expressões que cada classe especializada possui a definição de como pode interagir com as demais até chegar à última possibilidade de definição de literal.

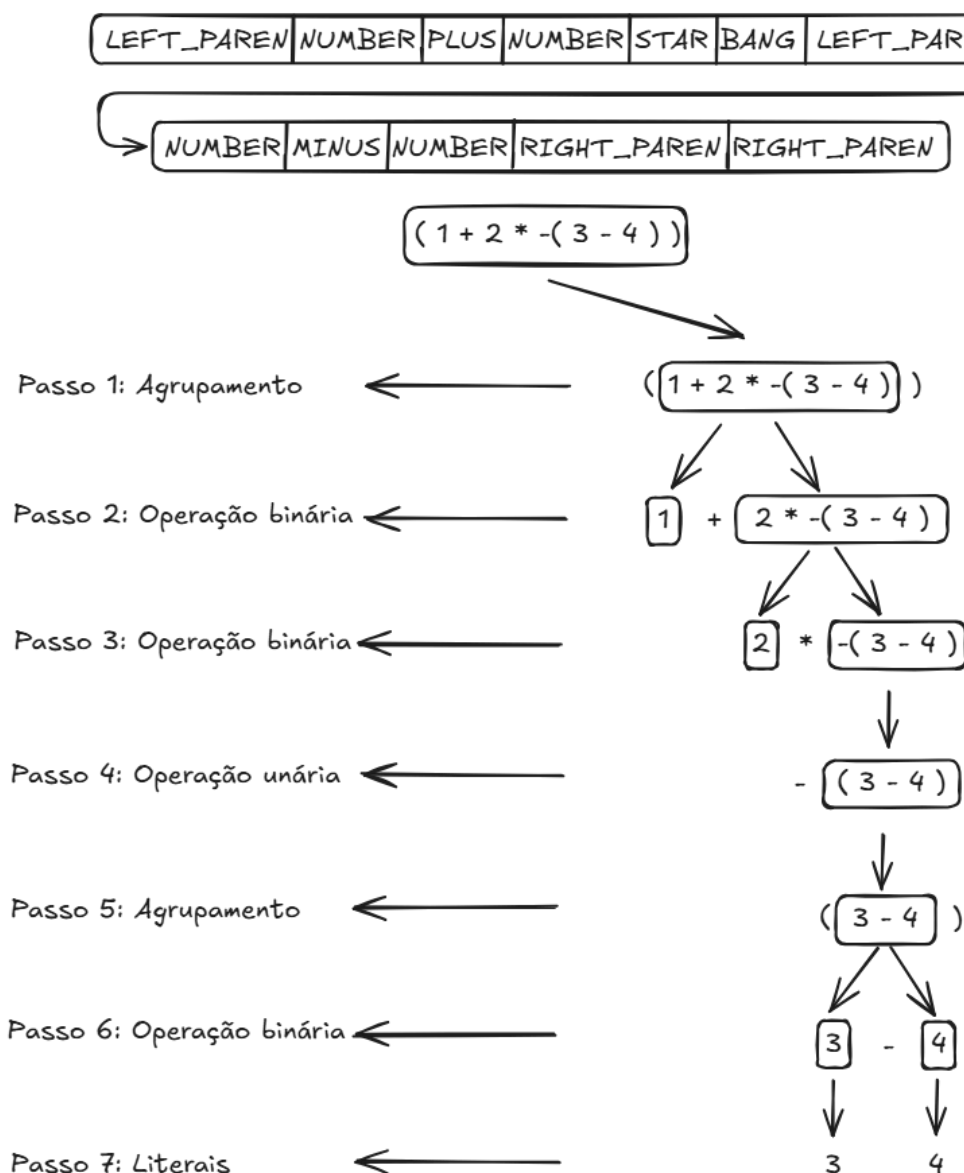


Figura 4. Exemplo do processo de "parser" de uma expressão.

Fonte: Dados originais da pesquisa

De volta para a Figura 3, que trata da hierarquia entre classes que representam objetos identificados pelo "parser", e detalhando as regras descritas para as classes, há

considerações importantes a serem feitas. O escopo definido para o projeto tange somente à hierarquia entre classes, logo não será avaliado o que está contido nos respectivos escopos. As regras definidas para o "parser" das classes, apresentadas na eq.3, se iniciam na declaração da classe, que tem como prefixo a palavra class seguida obrigatoriamente de um identificador e opcionalmente de uma base caso herde de alguma outra classe. E frente à restrição de avaliar somente a hierarquia de classes, tudo o que está contido dentro dos símbolos "{ }" é salvo como declaração de membro e não mais processado. A segunda regra é referente à classe base que acontece logo após o identificador da classe principal e tem seu prefixo de dois pontos ":". Não é especificado um número limite de classes base e por isso a segunda regra consiste apenas na replicação de especificações de classes base separadas por uma vírgula. A terceira regra já está relacionada com a definição da classe base, que precisa de uma especificação de tipo de acesso seguida de um identificador da classe base.

$$\begin{aligned} \text{Declaração de classe} &\rightarrow \text{"class" IDENTIFICADOR [":" declaração de base]} \\ &\quad \text{"{" declaração de membro "}} \\ \text{Declaração de base} &\rightarrow \text{especificador de base ("," especificador de base) } * \quad (3) \\ \text{Especificador de base} &\rightarrow \text{Especificador de acesso IDENTIFICADOR} \\ \text{Especificador de acesso} &\rightarrow \text{"public" | "private" | "protected"} \end{aligned}$$

Em que Declaração de classe corresponde à estrutura principal para definir uma classe, podendo conter um nome identificador, heranças opcionais e membros internos; Declaração de base representa a definição de classes das quais a classe atual herda, permitindo múltiplas bases separadas por vírgula; Especificador de base define a relação de herança com um modificador de acesso seguido do identificador da classe base; e Especificador de acesso corresponde ao nível de visibilidade da herança, podendo ser público, privado ou protegido.

Por fim, a terceira fase contempla a análise de contexto, que, no âmbito dos compiladores, é responsável por realizar um processamento adicional ao "parser", identificando erros funcionais com base na árvore de sintaxe. No contexto do projeto de pesquisa proposto, essa etapa é explorada de maneira limitada, restringindo-se à identificação de relações entre classes dentro do código analisado. Essa informação está contida no próprio "parser" da declaração das classes, e a coleta desses dados foi introduzida no fluxo já existente do "parser" para classes. Um container para armazenar informações dos objetos do tipo classe foi criado seguindo um formato de padrão de projeto

singleton, que será detalhado ainda nessa sessão, com a intenção de consolidar todas as informações das classes declaradas no código fonte avaliado.

O resultado desse processamento pode ser exportado em formato JSON, no qual cada classe identificada contém o seu identificador, que é chamado de `className`, e, caso aplicável, também contém as relações de herança associadas, com nome de `inherency`. A informação da herança necessita de duas informações: o tipo de acesso e o identificador da classe base, e são apresentados como primeiro e segundo itens contidos dentro do campo `inherency`. Na Figura 5 é apresentado um exemplo simplificado da saída para duas classes, onde B herda publicamente de A e, em formato JSON, temos os campos mencionados que mapeiam a relação entre as classes. E aqui está o resultado desejado ao final do "parser", dado que cada objeto de declaração de classe contém informações sobre seu identificador e a quem é relacionado hierarquicamente. Esse container de objetos de classes é então retornado no formato JSON para o requerente da API.

```
[
  {
    "className": "A",
    "inherency": []
  },
  {
    "className": "B",
    "inherency": [
      {
        "first": { "lexeme": "public", "type": "PUBLIC" },
        "second": { "lexeme": "A", "type": "IDENTIFIER" }
      }
    ]
  }
]
```

Figura 5. Exemplo da resposta da relação entre classes da API em formato JSON.

Fonte: Dados originais da pesquisa

Diante do contexto apresentado e do panorama funcional do software, é pertinente discutir sua implementação. A Figura 6 apresenta, em alto nível, o funcionamento da API que tem por fonte de dados uma base de código, em C++, e o resultado obtido é uma estrutura de dados no formato JSON contendo a relação de hierarquia entre as classes. De forma detalhada, o projeto pode expandir para suportar várias linguagens de programação, mapear diversos tipos de "tokens" para cada uma das linguagens e executar diversos tipos de análises estáticas e código dependendo do token e da linguagem selecionada.

O primeiro passo para desvendar a caixa preta da implementação da API é as especificações de uso. O escopo deste projeto de pesquisa limita-se à análise de códigos escritos em C++, extraídos de repositórios no GitHub, considerando somente estruturas

hierárquicas de classes. Entretanto, a escalabilidade para suportar outros tipos de linguagens e análises é considerada na arquitetura do software, que contempla as possíveis expansões para demais linguagens sem a necessidade de refatorações significativas na lógica principal, somente a adição de novas implementações na infraestrutura já existente.



Figura 6. Apresentação em alto nível do funcionamento da API.

Fonte: Dados originais da pesquisa

Adentrando nos detalhes de implementação, algumas guidelines e padrões de projetos da programação foram utilizados como referência para solucionar problemas e/ou auxiliar na arquitetura da solução. Tanto Meyers (2014), focado nas diretrizes de implementação em C++, quanto Iglberger (2022), com maior atenção aos padrões de projeto, exerceram grande influência nas decisões de implementação desse projeto. Esta influência consiste em diretrizes que não têm a finalidade de solucionar todos os problemas, mas quando adaptadas de acordo com o contexto, reduzem muito o esforço de desenvolvimento e o custo computacional. A ideia aqui é detalhar alguns dos padrões utilizados, com foco nos que mais agregaram no desenvolvimento do projeto.

O primeiro grande padrão utilizado foi o singleton, que está presente tanto na infraestrutura de logs quanto no container que armazena a relação final entre as classes. Apesar de amplamente questionado quanto à testabilidade, presença de estado global compartilhado e um acoplamento implícito entre diversas porções do software, o padrão singleton oferece uma grande simplicidade no controle de acesso, como em arquivos, garantindo que apenas uma instância esteja ativa. Iglberger (2022) questiona sobre os motivos desse padrão, originalmente classificado como padrão de projeto, ser caracterizado como padrão de implementação, por ser uma maneira muito eficaz de limitar o número de instâncias de um objeto, foco em implementação, do que desacoplar ou gerenciar dependências, foco em projeto.

A escolha do singleton se alinha justamente com esse controle de acesso a um recurso. Os logs que consistem na escrita em arquivo e, caso não tenham o padrão,

resultam em múltiplas chamadas para abrir, escrever e fechar o arquivo em diversas partes da implementação. Um grande risco de corromper o arquivo caso um procedimento comece antes do outro terminar, com somente uma instância responsável por isso, mitiga o risco de problemas. Já no caso dos containers com as relações entre as classes, o uso do singleton foi motivado para centralizar o armazenamento das informações em uma única instância, prevenindo assim o armazenamento compartilhado das informações que demandaria um pós-processamento para agrupá-los. Em um primeiro momento, parece simples controlar esse acesso, mas a complicação vem quando é preciso garantir a consistência da informação para múltiplos acessos e em locais dispersos pelo código.

O padrão de projeto visitor é outro exemplo empregado neste trabalho. Este aborda o antagonismo entre a inserção de novos tipos e a inclusão de novas operações em uma lógica implementada conforme orientação a objetos. Iglberger (2022) discorre sobre o tema com análises sobre a escolha do paradigma de programação frente à necessidade de adicionar novos tipos ou operações. Ao passo que em uma programação procedural seja simples adicionar operações novas, adicionar tipos tem por consequência alterar componentes base do código e propagar a mudança para as demais camadas que herdam dessa base. Em contrapartida, a programação orientada a objetos simplifica a adição de novos tipos por simplesmente adicionar mais uma classe que herda de uma base, mas complica a inserção de novas operações, por obrigar todas as bases a implementarem especializações para esta operação.

O padrão visitor atua nesse contexto para equilibrar essas limitações, delegando a implementação de novas operações a uma classe externa, que é "aceita" pela classe principal. O visitor convencional transforma o estilo de programação orientado a objetos de simples inserções de tipos para difícil e de difícil inserção de operação para simples. Em termos práticos, inverte a dificuldade. Mas existe uma alternativa que simplifica ambas as inclusões, o `std::variant` do C++. Tal alternativa reserva em memória o espaço necessário para o maior tipo que o visitor pode operar e utiliza esse bloco de memória para definir a ação e a classe que será executada. O padrão visitor também é abordado por Nystrom (2020) no contexto do "parser", ressaltando a simplicidade de adicionar novas operações em um conjunto de tipos específicos sem alterar a implementação da classe em questão.

No presente projeto, o visitor foi utilizado junto à análise recursiva das regras gramaticais definidas. As classes especializadas, como no exemplo das expressões, somente precisam declarar sua estrutura e métodos relacionados à obtenção dessas informações. Operações adicionadas como apresentar no console a ordem de processamento, função comumente conhecida como `prettyprint()`, podem e foram feitas por meio de visitors nas classes especializadas.

Outro padrão amplamente utilizado na implementação do projeto foi o strategy, também discutido por Iglberger (2022). Este padrão define uma família de algoritmos semelhantes, encapsulando cada um e tornando-o intercambiável conforme o tipo do contexto em que é utilizado. O ponto central está na utilização de uma interface, para que quando uma classe contexto precise de uma implementação específica, dependa apenas da interface e não da classe concreta, reduzindo assim o acoplamento entre as diferentes classes do projeto.

Uma analogia simples é um software responsável por desenhar formas geométricas, seja círculo, quadrado ou triângulo. Nesse caso, a estrutura proposta implementa uma interface com a função desenhar, enquanto cada classe especializada implementa o comportamento específico. Assim, ao utilizar a interface, o software resolve em tempo de execução qual implementação do método desenhar deve ser executada frente ao contexto que fez a requisição. Promovendo, assim, maior flexibilidade e desacoplamento na implementação da classe contexto com a classe concreta.

No projeto, o padrão strategy foi utilizado no tratamento de URLs com a biblioteca curl. Embora esta biblioteca tenha sido utilizada, qualquer outra poderia substituí-la, desde que respeite o mesmo contrato de entrada e saída, possibilitando alteração de dependências sem necessidade de refatoração de todo o fluxo, classes de contexto que utilizam URLs. Além disso, este padrão mostrou-se essencial na implementação dos testes e na injeção de dependências e na criação de objetos falsos (mocks) para simular comportamentos externos. Essa abordagem proporcionou maior controle sobre os cenários de teste, reduziu a complexidade e aumentou a confiabilidade do processo de validação por meio da criação dos testes unitários.

Ainda no exemplo das URLs, a Figura 7 apresenta esse padrão em ação. A implementação inicial contemplou uma interface para estabelecer o contrato com a classe concreta, responsável pelo tratamento das informações da URL. Posteriormente, foi realizado um experimento de alterar a lógica interna das funções. Para isso, bastou criar uma nova classe concreta GitHubAPIUrlInfo, implementar os testes unitários para validar seu funcionamento e atualizar a classe que a interface iria usar. O mesmo comportamento foi garantido por meio do contrato já estabelecido, mantendo as entradas e saídas de dados inalteradas, de forma a não necessitar alterações nas classes de contexto que utilizam essa interface.

Entre as diversas ferramentas disponíveis para a implementação do projeto, a linguagem C++ foi selecionada pela performance e eficiência. A estrutura do software em formato de uma API RESTful segue a tendência de oferecer praticidade ao usuário na utilização. Sem a necessidade de transferir ou instalar localmente, é possível simplesmente

interagir com o serviço e obter a informação desejada. O resultado disponibilizado em formato JSON foi escolhido por ser amplamente adotado em APIs e por possibilitar conversão simples para diversas aplicações. A biblioteca, em C++, utilizada para implementação da estrutura em API foi a Crow, cuja escolha fundamentou-se no suporte ativo da ferramenta e na qualidade da documentação fornecida.

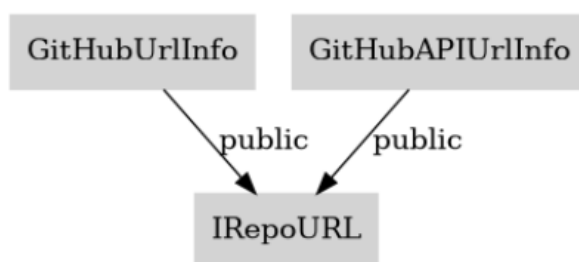


Figura 7. Exemplo do padrão de projeto Strategy.

Fonte: Dados originais da pesquisa

Frente ao projeto proposto, foi fundamental associar a implementação de testes ao desenvolvimento do projeto para assegurar a qualidade e confiabilidade do software. Foi pertinente desenvolver dois tipos de testes: unitários e de integração. No total, foram implementados 317 testes unitários e 18 testes funcionais, sendo os dados apresentados na seção de resultados derivados destes mesmos testes, visto que permitem simular um uso real da API ao acessar os mesmos endpoints que um usuário acessa.

Os testes unitários foram utilizados na validação de componentes da implementação, de forma isolada, explorando as particularidades e detalhes das lógicas internas. O framework escolhido para os testes unitários é o Google Tests (gtest), amplamente adotado na indústria e com vasta documentação. Para sua elaboração, seguiu-se a metodologia do padrão Test-Driven Development (TDD) abordado por Langr (2013), que ilustra os conceitos utilizando o framework em C++ e propõe que os testes sejam definidos antes ou em paralelo com a implementação da lógica funcional. Essa abordagem garante que cada componente da lógica seja testado de forma independente, sem influência externa, e prevenir regressões durante o desenvolvimento, dado que um teste funcional não pode falhar frente a uma nova implementação, com a exceção da mudança esperada de comportamento. Essa prática foi essencial para garantir a confiabilidade da implementação frente ao contexto do "scanner" e do "parser", nos quais pequenas alterações podem gerar impactos significativos no comportamento do sistema.

Apesar de sua relevância, os testes unitários não são suficientes para validar a completude do algoritmo. Por esse motivo, foram complementados com testes de

integração, que permitem avaliar o comportamento do software em uma perspectiva de alto nível. Este modelo de teste, também chamado de testes funcionais, consiste em prover entradas já conhecidas ao software, chamar o endpoint correspondente ao teste que deseja ser realizado nas APIs e avaliar o retorno obtido com o esperado para aquela determinada entrada. No contexto desse projeto, os testes funcionais foram responsáveis por gerar os dados apresentados na sessão de resultados, em que diversos exemplos foram adicionados ao repositório GitHub e suas respectivas URLs foram fornecidas à API com o intuito de gerar a hierarquia entre classes e fornecer conteúdo para as discussões e análise dos resultados.

Essa combinação estratégica de testes explorou a granularidade dos testes unitários com a abrangência dos testes funcionais, assegurando maior qualidade do software. Limitações e correções não previstas inicialmente puderam ser identificadas e solucionadas ao longo do desenvolvimento, sem comprometer o andamento do projeto. Além disso, a ampla base de testes ofereceu uma camada de segurança adicional: novas implementações só eram incorporadas ao projeto caso não provocassem regressão funcional nos testes já existentes. Essa prática reforçou a integração contínua e aumentou a confiabilidade do sistema ao longo do desenvolvimento.

Em síntese, a metodologia da implementação contemplou desde a infraestrutura necessária para entrada do código fonte para análise, definição da arquitetura do software e escolha de padrões de projeto até a implementação orientada a testes. A sessão seguinte apresenta os resultados obtidos a partir da aplicação prática dessa metodologia. Os conceitos apresentados são discutidos por meio de exemplos que abrangem desde a extração de estruturas sintáticas até a representação das relações entre as classes em formato JSON e convertidas para o formato DOT para uma representação gráfica.

Resultados e Discussão

A apresentação dos resultados segue um modelo incremental, com o intuito de expor progressivamente cada etapa do processo, de forma a simplificar a compreensão do leitor na análise dos resultados funcionais. Dado um código fonte de entrada, a API implementada apresenta a hierarquia de classes correspondente. O modelo da Figura 8 foi implementado para detalhar cada etapa do processo. A implementação utilizada como entrada de dados para a API é definida por uma classe base “Animal”, herdada por três outras classes: “Cachorro”, “Gato” e “Rato”. O processamento, como mencionado na seção anterior, foca somente na relação de herança entre as classes. Os métodos internos são registrados como declarações pertencentes à classe, mas não são processados ou retornados nos resultados. Dessa forma, a API tem por foco principal a relação hierárquica entre as classes,

apresentada na Figura 8, que exibe tanto o código utilizado quanto o resultado esperado da API.

```
// Example.cpp
class Animal {
public:
    void speak();
};

class Cachorro : private Animal {
public:
    void wagTail();
};

class Gato : public Animal {
public:
    void purr();
};

class Rato : protected Animal {
public:
    void squeak();
};
```

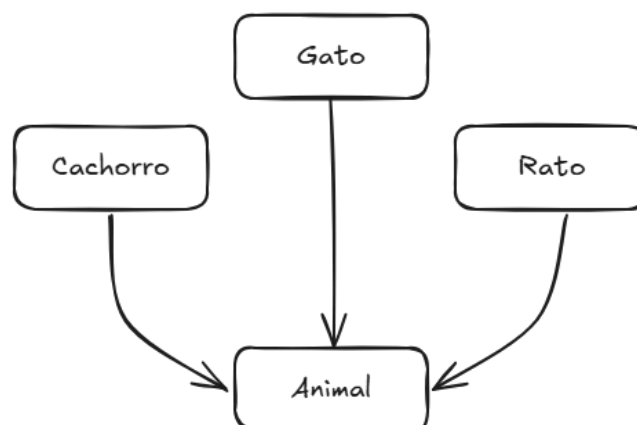


Figura 8. Código exemplo do processo de mapeamento das heranças entre classes.

Fonte: Resultados originais da pesquisa

Para ser processado pela API, o código deve estar hospedado em um repositório GitHub e, para esse exemplo, o código pode ser acessado neste [link](#). Dado que a API está em pleno funcionamento, é possível acessar o endpoint `/api/v1/processSourceCode` com a respectiva URL do teste proposto, como argumento, e iniciar a identificação da relação entre as classes.

A primeira etapa consiste na extração de informações com base na URL fornecida, necessária para realizar o download dos arquivos a serem analisados. A transferência dos arquivos é realizada por meio de uma API fornecida pelo próprio GitHub. A Figura 9 apresenta a conversão entre URLs, de acesso ao arquivo no repositório para a construção da requisição de download desse mesmo arquivo. As informações de interesse na URL de acesso são: autor do projeto, o repositório que hospeda o projeto, a respectiva ramificação do projeto e a relação de pastas que contêm o arquivo. Esse processo de identificação é realizado por uma expressão regular, apresentada na eq.4, que identifica as variáveis e retorna cada elemento. Após coletados os dados, é realizada a construção da URL de download dos arquivos.

`https://github\.com\[^\^]+\V([^\^]+)\V(tree|blob)\V([^\^]+)\V(.*)?)` (4)

Em que cada grupo de captura tem um papel específico: `([^\^]+)` é a identificação do autor; a segunda ocorrência de `([^\^]+)` é a identificação do repositório; `(tree|blob)` é a distinção se a URL refere a um diretório (`tree`) ou a um arquivo (`blob`); a terceira ocorrência de `([^\^]+)` é a identificação da ramificação do repositório; e `\V(.*)?)` captura de forma opcional o caminho completo do arquivo ou pasta avaliado.arquivos.

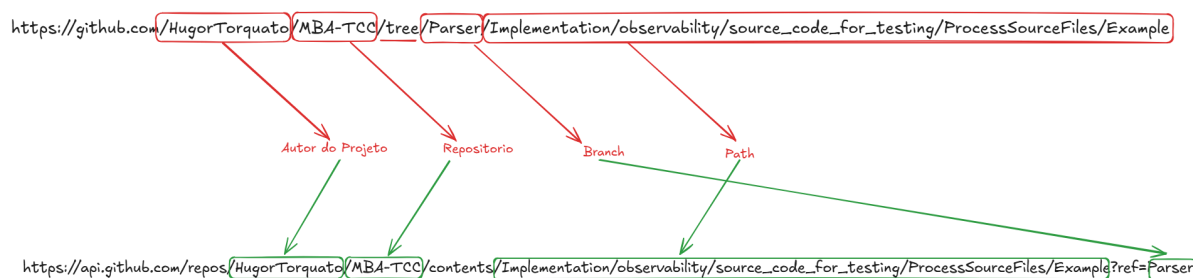


Figura 9. Conversão entre URL de acesso e URL de download de arquivo GitHub.

Fonte: Resultados originais da pesquisa.

Os arquivos contidos no caminho fornecido e os respectivos filhos são identificados, listados e transferidos para uma pasta temporária. Para manter o contexto de origem dos arquivos, foi implementada uma estrutura em grafos que armazena a organização dos diretórios. No exemplo da Figura 8, a estrutura contém apenas um nó, correspondente ao arquivo fonte. A Figura 10 ilustra o conteúdo obtido pela API do GitHub, com diversas meta informações sobre o arquivo sendo, as mais relevantes para este projeto, o nome e caminho do arquivo, sua respectiva url de acesso e para download e o tipo que este item representa, no caso um arquivo. Esse resultado pode ser acessado por meio deste [link](#).

```
[
  {
    "name": "Example.cpp",
    "path": "Implementation/observability/source_code_for_testing/ProcessSourceFiles/Example/Example.cpp",
    "sha": "81b97a896d7d918ffc3d1f85a500da1811e28770",
    "size": 245,
    "url": "https://api.github.com/repos/HugorTorquato/MBA-TCC/contents/Implementation/observability/source_code_for_testing/ProcessSourceFiles/Example/Example.cpp?ref=Parser",
    "html_url": "https://github.com/HugorTorquato/MBA-TCC/blob/Parser/Implementation/observability/source_code_for_testing/ProcessSourceFiles/Example/Example.cpp",
    "git_url": "https://api.github.com/repos/HugorTorquato/MBA-TCC/git/blobs/81b97a896d7d918ffc3d1f85a500da1811e28770",
    "download_url": "https://raw.githubusercontent.com/HugorTorquato/MBA-TCC/Parser/Implementation/observability/source_code_for_testing/ProcessSourceFiles/Example/Example.cpp",
    "type": "file",
    "_links": {
      "self": "https://api.github.com/repos/HugorTorquato/MBA-TCC/contents/Implementation/observability/source_code_for_testing/ProcessSourceFiles/Example/Example.cpp?ref=Parser",
      "git": "https://api.github.com/repos/HugorTorquato/MBA-TCC/git/blobs/81b97a896d7d918ffc3d1f85a500da1811e28770",
      "html": "https://github.com/HugorTorquato/MBA-TCC/blob/Parser/Implementation/observability/source_code_for_testing/ProcessSourceFiles/Example/Example.cpp"
    }
  }
]
```

Figura 10. Meta informações de cada URL de acesso a arquivos no GitHub.

Fonte: Resultados originais da pesquisa

A próxima etapa contempla a leitura dos arquivos e a materialização do conteúdo em uma estrutura de dados palpável para a etapa de "scanner". Cada linha é carregada e armazenada como string. Para garantir organização e rastreabilidade, as informações são armazenadas em uma estrutura JSON contendo o arquivo, caminho e conteúdo em formato string. A Figura 11 apresenta o JSON resultante ainda para o exemplo da Figura 8, contendo as informações do arquivo seguidas de seu conteúdo. Esse exemplo contempla somente um único arquivo, mas esse padrão é propagado para todos os arquivos pertencentes ao diretório especificado na API e suas ramificações. Neste caso, um JSON extenso é gerado e contempla todos os arquivos pertencentes ao selecionado diretório no GitHub e suas respectivas informações. Esta etapa comprova que a coleta de dados está correta e precede à análise léxica.

```
{
  "Example.cpp:Implementation/observability/source_code_for_testing/ProcessSourceFiles/Example/Example.cpp":
  "// Example.cpp\n\\class Animal {\n\\public:\n    void speak();\n};\n\\n\\n\\class Dog : public Animal {\n\\public:\n    void wagTail();\n};\n\\n\\n\\class Cat : public Animal {\n\\public:\n    void purr();\n};"
```

Figura 11. Resultado no formato JSON da relação entre arquivo e conteúdo.

Fonte: Resultados originais da pesquisa

De posse do conteúdo de cada arquivo, inicia a etapa de "scanner". Esta percorre caractere a caractere, de forma a emitir "tokens" e armazená-los em um container que será utilizado na etapa posterior, o "parser". Ainda no exemplo da Figura 8, a ferramenta reconhece: palavras-chave como class, public, void; identificadores de classes e métodos como Animal, Cachorro, Gato, Rato; símbolos de estrutura como "{", "}", "(", ")", ";", e a pontuação que sinaliza herança ":". Essa granularidade é registrada pelos logs, que registram cada token, seu respectivo lexema e posição no código fonte, linha e coluna. De forma a expor todo o processo, a Tabela 1, que está em apêndice, agrupa os "tokens" gerados para o caso de exemplo junto das informações coletadas. Embora verbosa, essa coleta de informações é essencial, já que as regras gramaticais necessitam do mapeamento dos "tokens" para serem efetivas.

A etapa do "parser" transforma a sequência de "tokens", armazenada durante o "scanner", em uma estrutura hierárquica que representa parte da gramática da linguagem de programação alvo, no caso, o C++. Essa é a estrutura chamada de árvore de sintaxe abstrata. Para este projeto, a proposta é identificar classes e relações de herança entre elas. Logo, as classes e subclasses foram identificadas e o resultado foi organizado na estrutura em árvore. Cada nó representa uma classe e contém informações sobre herança que foram

definidas no código fonte, e as arestas definem o tipo de relação entre as classes. A Figura 12 apresenta parte desse processo, listando as etapas realizadas até a construção da AST partindo do código fonte, "scanner", "parser" e, por fim, a árvore de sintaxe.

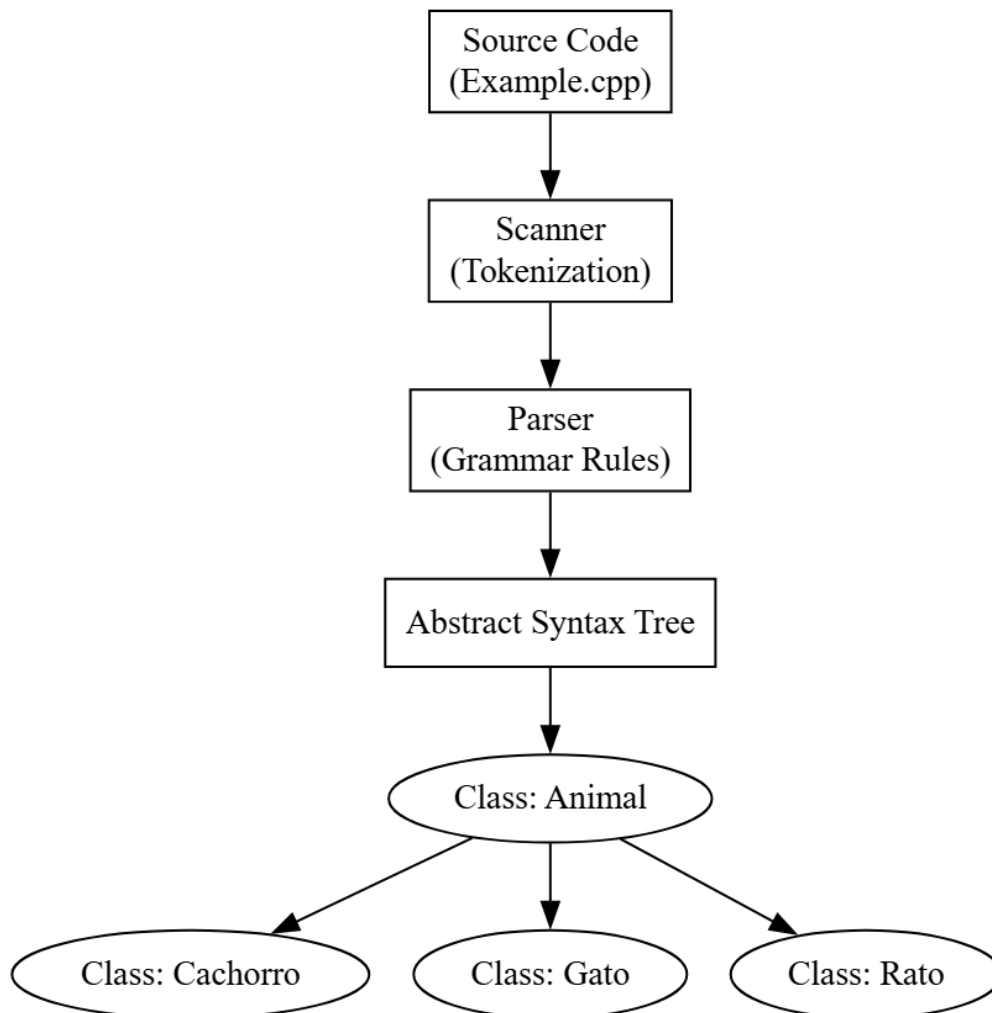


Figura 12. Etapas detalhadas até o momento para o exemplo da Figura 8.

Fonte: Resultados originais da pesquisa

Por fim, o resultado do exemplo da Figura 8, fornecido pela API desenvolvida, é um arquivo JSON que contém a relação entre as classes. A Figura 13 contempla tanto a resposta no formato JSON quanto a conversão para o formato DOT, uma representação gráfica que auxilia na visualização. A Figura 13 apresenta as informações do JSON no formato DOT, a definição do grafo em texto e a respectiva visualização gráfica. No diagrama, é possível destacar o tipo de relação entre as classes, sendo vermelho para relação privada, preto para pública e azul para protegida. Todas com herança direta para a classe base "Animal".

A Figura 13 concluiu o processo de identificação entre as classes do exemplo dos animais. Agora, todo o processo contido na caixa preta da API, apresentada na Figura 6, foi explicado e exemplificado com o caso teste apresentado. Agora, não há somente um exemplo nem uma relação direta entre classes. Os próximos exemplos focam em apresentar, de forma didática, a API em funcionamento. A intenção é demonstrar a consistência na identificação das classes diante de casos frequentes na implementação orientada a objetos. O primeiro exemplo consiste na identificação de uma definição de classe sem herança, caso base de declaração de classes, e que é fundamental garantir seu funcionamento.

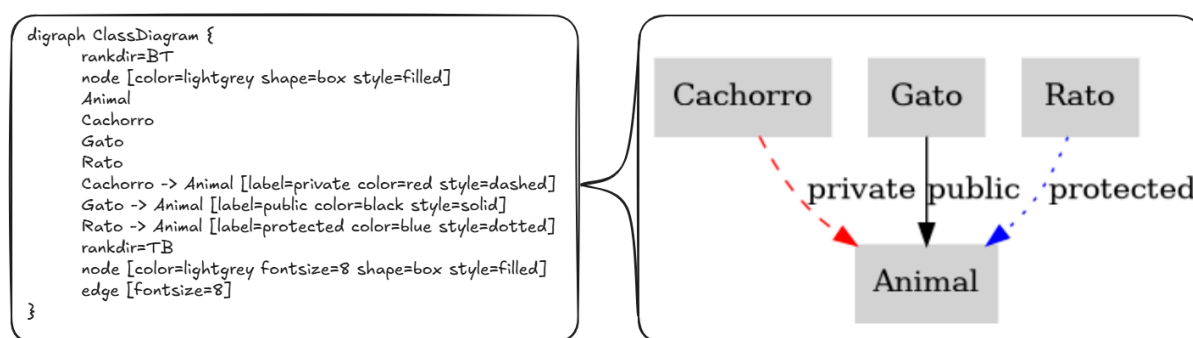


Figura 13. Conversão do resultado no formato DOT para diagrama ilustrativo.

Fonte: Resultados originais da pesquisa

A Figura 13 concluiu o processo de identificação entre as classes do exemplo dos animais. Agora, todo o processo contido na caixa preta da API, apresentada na Figura 6, foi explicado e exemplificado com o caso teste apresentado. Agora, não há somente um exemplo nem uma relação direta entre classes. Os próximos exemplos focam em apresentar, de forma didática, a API em funcionamento. A proposta é demonstrar a consistência na identificação das classes diante de casos frequentes na implementação orientada a objetos. O primeiro exemplo consiste na identificação de uma definição de classe sem herança, caso base de declaração de classes, e que é fundamental garantir seu funcionamento.

O exemplo em código quanto ao diagrama gerado pela conversão do resultado para o formato DOT, apresentado já no modo gráfico, é possível ser visto na Figura 14. A classe tem por identificador o nome “C1” e é apresentada no diagrama de blocos sem nenhuma conexão externa.

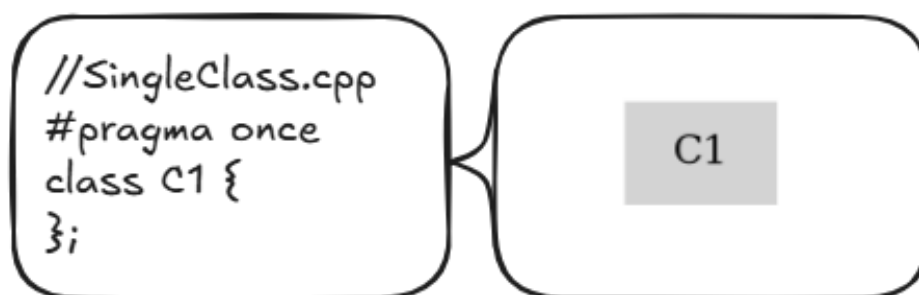


Figura 14. Exemplo de declaração de classe sem herança.

Fonte: Resultados originais da pesquisa

Por sua vez, a Figura 15 confirma que a API é capaz de identificar herança pública simples entre classes, em que A herda publicamente de B. Esse padrão é a base de identificação da relação de classes em qualquer projeto que utilize essa estratégia.



Figura 15. Exemplo de declaração de classe com herança pública simples.

Fonte: Resultados originais da pesquisa

A Figura 16 direciona a análise para validar a capacidade do "parser" em identificar o caso de heranças múltiplas mistas, com diferentes níveis de acesso na mesma definição de classe. Esse arranjo pode ser útil para reutilizar funcionalidades de forma seletiva, mas também aumenta a complexidade de manutenção. Análogo ao exemplo anterior, foi definida uma herança pública entre as classes C e A, mas também foi definida uma herança privada entre as classes C e B. Esta é representada por uma seta pontilhada vermelha.

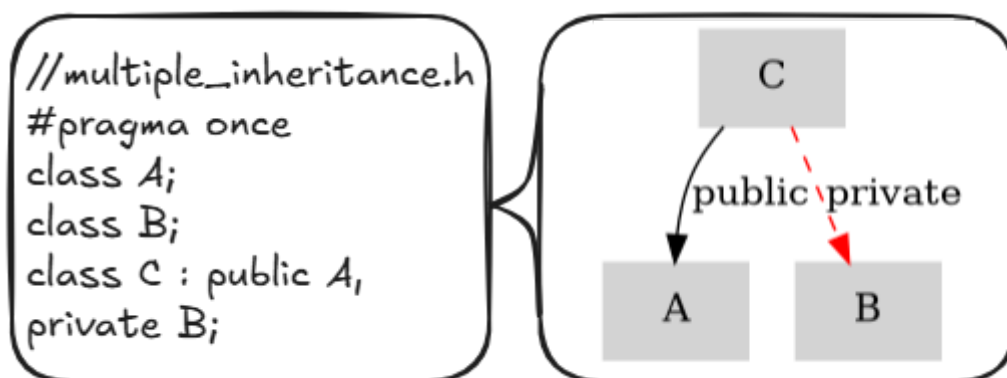


Figura 16. Exemplo de declaração de classe com herança múltipla mista.

Fonte: Resultados originais da pesquisa

Na Figura 17 é contemplada a relação entre classes, mas com definição em arquivos diferentes. A API tem de ser resiliente e manter a correta correlação entre as classes independentemente de serem definidas em arquivos distintos, o que contempla a maioria dos projetos com programação orientada a objetos. No exemplo apresentado, uma classe “ble” é definida no arquivo “classDef.h” mas somente utilizada no arquivo “ClassDef2.h” como base da classe “bla”. O diagrama apresenta a correta relação de herança pública simples entre as classes, independentemente de serem definidas em arquivos diferentes.



Figura 17. Exemplo de declaração com herança pública simples de diferentes arquivos.

Fonte: Resultados originais da pesquisa

A Figura 18 apresenta o clássico problema do diamante na programação orientada a objetos, em que tanto classes B e C herdam de A quanto a classe D herda de ambos B e C em uma herança múltipla. É um problema que gera ambiguidade com múltiplas cópias da classe base. No C++, esse problema é solucionado com a herança virtual, que ainda não é

suportada no presente projeto. Não temos falhas no processo de "parser" para funções virtuais, mas o identificador "virtual" não é associado à identificação da herança.

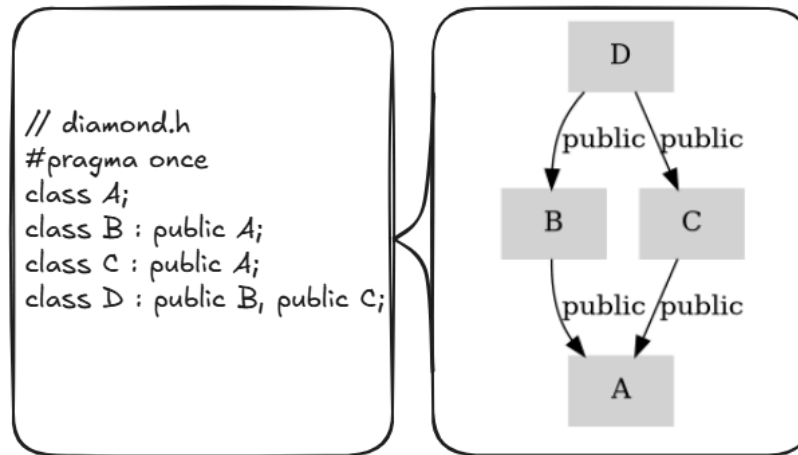


Figura 18. Exemplo clássico do problema da herança diamante.

Fonte: Resultados originais da pesquisa

Retomando os exemplos mais usuais, a Figura 19 apresenta uma cadeia linear de herança, em que a classe B herda com acesso público da classe A, na sequência C herda de B e D herda de C, de forma a criar uma cadeia linear de derivações. O principal uso dessa arquitetura é nas especializações, em que essa herança em cadeia linear representa um determinado ramo do todo.

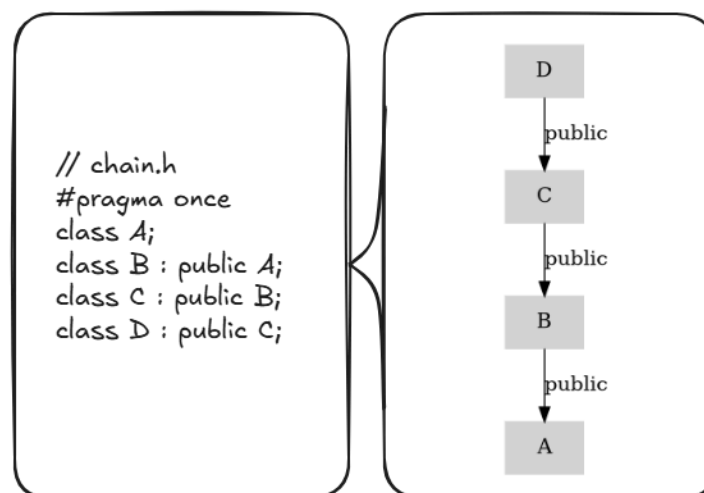


Figura 19. Exemplo clássico de herança em cadeia linear.

Fonte: Resultados originais da pesquisa

Os resultados até agora focaram em exemplos didáticos com relações diretas entre as classes. Contudo, a API desenvolvida também é capaz de analisar arquiteturas maiores

como um zoológico, por exemplo. Este código possui tanto a relação entre a cadeia de classificação biológica dos animais quanto o gerenciamento de recursos financeiros. Dada uma implementação genérica, temos como resposta da API o diagrama da Figura 20.

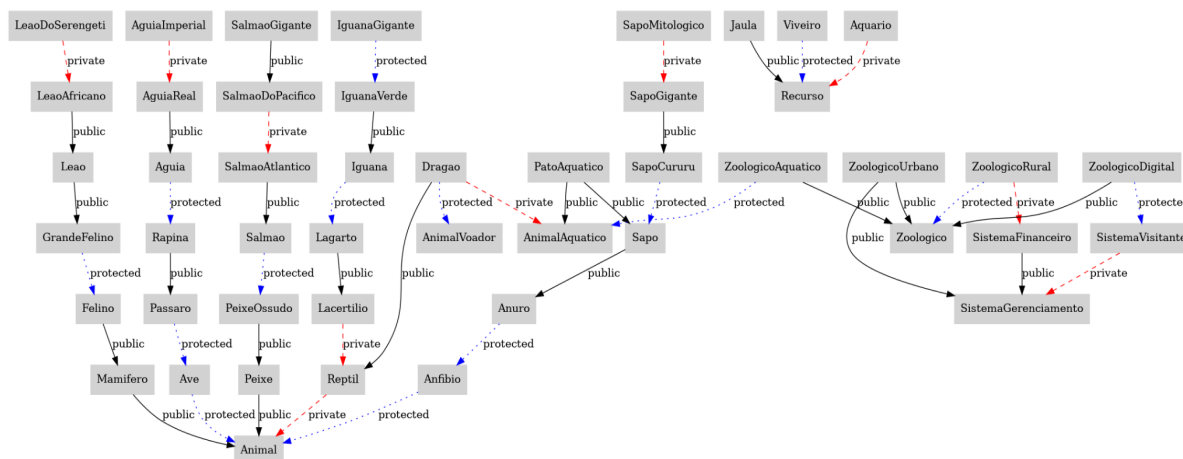


Figura 20. Exemplo da hierarquia entre classes para um roteiro temático a um zoológico.

Fonte: Resultados originais da pesquisa

Nota: A figura pode ser acessada no [link](#)

A quantidade de informações torna a Figura 20 pouco legível, mas o diagrama completo pode ser acessado pelo [link](#) ao repositório do projeto, para melhor visualização. Ainda assim, a imagem representa toda a relação entre classes do exemplo do zoológico. Com foco na relação de custos, a Figura 21 apresenta uma versão reduzida da relação entre as classes. Nesse modelo, o recurso é utilizado tanto para manutenção quanto para passivos do zoológico, mas ao mesmo tempo também é destinado a pagar os funcionários que são divididos em subclasses conforme sua profissão. Sem um significado funcional específico, a relação entre as classes foi ajustada para exercitar a criação dos diferentes tipos de relações entre classes, representadas pelas setas de diferentes cores.

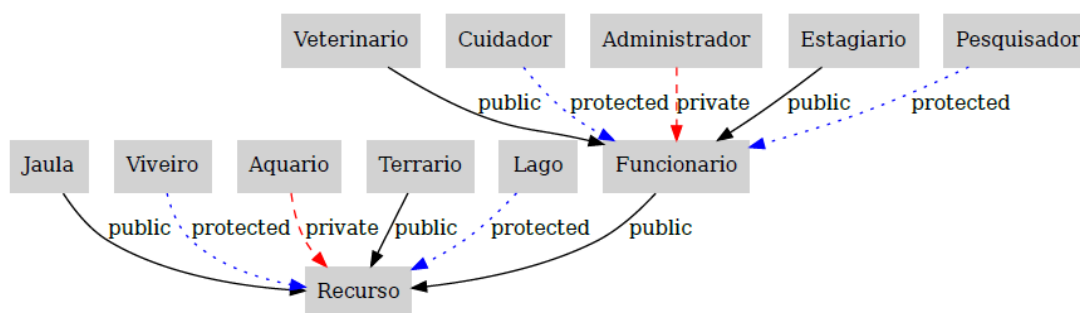


Figura 21. Exemplo, reduzido, da hierarquia entre classes para um zoológico.

Fonte: Resultados originais da pesquisa

Apresentados os casos de exemplo em que a ferramenta é capaz de atuar, surge o questionamento: “Posso utilizar essa API no meu código?” A resposta é sim e o projeto está disponível publicamente nesse [link](#) no GitHub. Inclusive, as Figuras 3 e 7, expostas na sessão de materiais e métodos, foram geradas por meio da API desenvolvida neste trabalho. Estas foram utilizadas para explicação da hierarquia entre classes dos elementos do “parser” e a explicação do uso do padrão de projeto strategy, respectivamente.

Os resultados qualitativos dos testes, no entanto, destacam um ponto de atenção, que também é um ponto de melhoria do projeto: o tempo consumido por análise é significativo frente ao tamanho do projeto. Para projetos pequenos, como os primeiros exemplos apresentados nesta sessão, o tempo médio de execução variou entre 1 e 3 segundos para a iteração de ponta a ponta com a API, que é compreensivo frente aos processos que têm de ser executados. Entretanto, para exemplos mais elaborados, como o do zoológico, a medição média foi de 30 segundos. Ainda é considerado um projeto pequeno, mas é possível notar significativa variação no tempo gasto. Porém, o caso mais relevante, e que se aproxima das implementações reais, é o próprio projeto implementado para este trabalho.

Em um contexto de ordem de tamanho, o projeto contém aproximadamente 14 mil linhas de código segregadas em 113 arquivos, sendo 52 no formato .h, 49 no formato .cpp e 12 no formato .py. Embora não possa ser considerado um projeto de grande porte, também não é trivial. Para essa proporção, foram necessárias aproximadamente 2 horas de processamento para analisar todos os arquivos e gerar a relação hierárquica entre as classes. Um salto expressivo de tempo que representa a maior limitação da usabilidade da ferramenta. A Figura 22 apresenta a completude do projeto desenvolvido, mas para melhor visualização pode ser acessada pelo [link](#) que contém a imagem no repositório GitHub. É importante considerar que o diagrama inclui tanto o código fonte da lógica de negócio quanto a implementação dos testes unitários e funcionais. Isso aumentou a densidade de informações no diagrama, mas também evidenciou a abrangência do projeto.

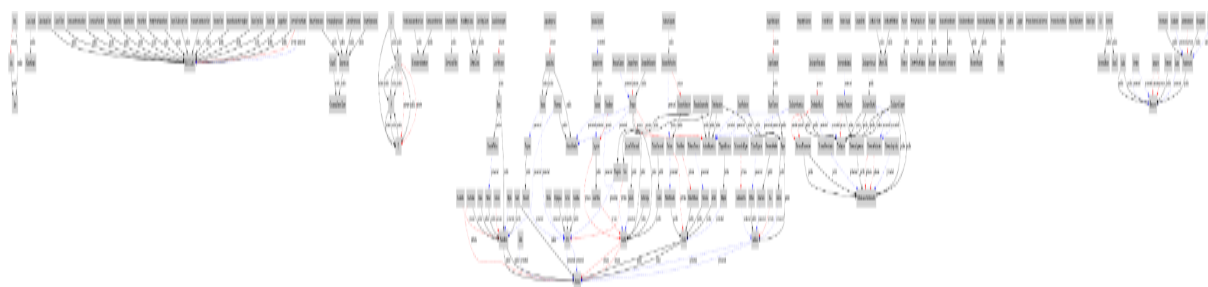


Figura 22. Diagrama com todas as classes implementadas no presente projeto.

Fonte: Resultados originais da pesquisa

Nota: A figura pode ser acessada no [link](#)

Em uma investigação inicial sobre a performance do projeto, foram identificados diversos pontos de melhorias, tais como: substituição do uso de strings por estruturas de dados mais especializadas; utilização de cache para armazenamento do código fonte, evitando operações repetidas de escrita e leitura em disco; refatoração do sistema de comunicação, atualmente baseado em chamadas diretas à API externas sem camadas intermediárias de controle ou balanceamento de carga; e, por fim, a atualização do hardware utilizado, já que tudo foi realizado em um notebook com mais de 10 anos de uso.

Considerações Finais

O presente trabalho teve por objetivo a implementação de uma API capaz de mapear a herança entre classes em projetos orientados a objetos na linguagem C++. Com isso, facilitar discussões, a tomada de decisão e maior praticidade da compreensão dos processos. Sendo assim, fez-se relevante arquitetar um software capaz de realizar uma análise estática em códigos baseados em programação orientada a objetos, extrair informação sobre a organização das classes e expor, de maneira simples e direta, informações estruturadas acerca do código em questão.

Frente aos resultados obtidos, o software demonstrou eficácia e precisão. Foi possível identificar padrões recorrentes em projetos, como classes sem herança, com herança simples, múltipla, distribuída em diferentes arquivos, além de herança em diamante e em cascata. A API também se mostrou capaz de processar projetos maiores, como o exemplo do zoológico e a própria implementação desenvolvida neste estudo.

O tempo de processamento se destacou como principal limitação. Em projetos pequenos, a resposta é obtida em segundos, mas em projetos maiores pode demandar horas. A investigação dessa questão e possíveis melhorias na arquitetura da API representam oportunidades relevantes para trabalhos futuros.

Além da implementação prática, o projeto possibilitou reflexões sobre temas relacionados a compiladores, arquitetura de software e boas práticas de implementação. Apesar do escopo restrito à análise de classes, o trabalho abre espaço para ampliações, como a coleta estática de outros elementos do código ou o suporte a novas linguagens de programação, bem como trabalhos futuros.

Agradecimento

Agradeço, primeiramente, a Deus, à minha família, à minha noiva que iniciou esta jornada como namorada e aos amigos, pelo apoio e compreensão constantes. O incentivo de cada um foi essencial para que eu tivesse motivação e dedicação na conclusão do MBA.

Referências

- Aho,A.V; Lam,M.S; Sethi, R; Ullman, J.D. 2007. Compilers: Principles, Techniques, & Tools. Segunda Edição. Pearson Education, Inc, Boston, Massachusetts, Estados Unidos.
- Ceconello, J. 2016. Ferramenta de auxílio à análise de anomalias em códigos Orientados a Objeto. Monografia em Ciência da Computação. Universidade Federal do Rio Grande do Sul, Porto Alegre, Rio Grande do Sul, Brasil.
- Cooper, K.D; Torczon,L. 2012. Engineering a Compiler. Segunda Edição. ELSEVIER, Huston, Texas, Estados Unidos.
- Dean,J; Grove,D; Chambers,C. 1995. Optimization of Object-Oriented Programs Using Static Class Hierarchy Analysis. 9th European Conference: 77-101.
- Iglberger, K. 2022. C++ Software Design: Design Principles and Patterns for High-Quality Software. O'Reilly Media, Inc, Sebastopol, Califórnia, Estados Unidos.
- Langr, J. 2013. Modern C++ Programming with Test-Driven Development. The Pragmatic Programmers, LLC. Colorado Springs, Colorado, Estados Unidos.
- Martin, R.C. 2009. Clean Code A Handbook of Agile Software Craftsmanship. Pearson Education, Inc, Boston, Massachusetts, Estados Unidos.
- Meyers, S. 2014. Effective Modern C++: 42 Specific Ways to Improve Your Use of C++11 and C++14. O'Reilly Media, Inc, Sebastopol, Califórnia, Estados Unidos.
- Nystrom, R. 2015-2020. Crafting Interpreters. Genever Benning. Seattle, Washington, Estados Unidos.
- Silva, D; Samarasekara, H.M.P.P.K.H; Hettiarachchi, R.T; Senanayake, W.A.B.M; Sanjaya, A.M.T; Abeysekara, M.P. 2023. A Comparative Analysis of Static and Dynamic Code Analysis Techniques. TechRxiv: DOI: 10.36227/techrxiv.22810664.v1.
- Wang, S; Ding, L; Shen, L; Luo, Y; Du, B; Tao, D. 2024. Título do trabalho. Findings of the Association for Computational Linguistics: 13619–13639.

Apêndice

Tabela 1. Lista de "tokens" gerados para o exemplo apresentado na Figura 8.

(continua)

Lexeme	TokenType	Line	Column
class	CLASS	2	1
Animal	IDENTIFIER	2	7
{	LBRACE	2	14
public	ACCESS	3	1
:	COLON	3	7
void	IDENTIFIER	4	5
speak	IDENTIFIER	4	10
(	LPAREN	4	15
)	RPAREN	4	16
;	SEMICOLON	4	17
}	RBRACE	5	1
;	SEMICOLON	5	2
class	CLASS	7	1
Cachorro	IDENTIFIER	7	7
:	COLON	7	16
private	ACCESS	7	18
Animal	IDENTIFIER	7	26
{	LBRACE	7	33
public	ACCESS	8	1
:	COLON	8	7
void	IDENTIFIER	9	5
wagTail	IDENTIFIER	9	10
(	LPAREN	9	17
)	RPAREN	9	18
;	SEMICOLON	9	19
}	RBRACE	10	1
;	SEMICOLON	10	2
class	CLASS	12	1
Gato	IDENTIFIER	12	7
:	COLON	12	12
public	ACCESS	12	14
Animal	IDENTIFIER	12	21
{	LBRACE	12	28
public	ACCESS	13	1
:	COLON	13	7
void	IDENTIFIER	14	5
purr	IDENTIFIER	14	10
(	LPAREN	14	14
)	RPAREN	14	15

Tabela 1. Lista de "tokens" gerados para o exemplo apresentado na Figura 8.

(conclusão)

Lexeme	TokenType	Line	Column
;	SEMICOLON	14	16
}	RBRACE	15	1
;	SEMICOLON	15	2
class	CLASS	17	1
Rato	IDENTIFIER	17	7
:	COLON	17	12
protected	ACCESS	17	14
Animal	IDENTIFIER	17	24-
{	LBRACE	17	31
public	ACCESS	18	1
:	COLON	18	7
void	IDENTIFIER	19	5
squeak	IDENTIFIER	19	10
(	LPAREN	19	16
)	RPAREN	19	17
;	SEMICOLON	19	18
}	RBRACE	20	1
;	SEMICOLON	20	2

Fonte: Resultados originais da pesquisa